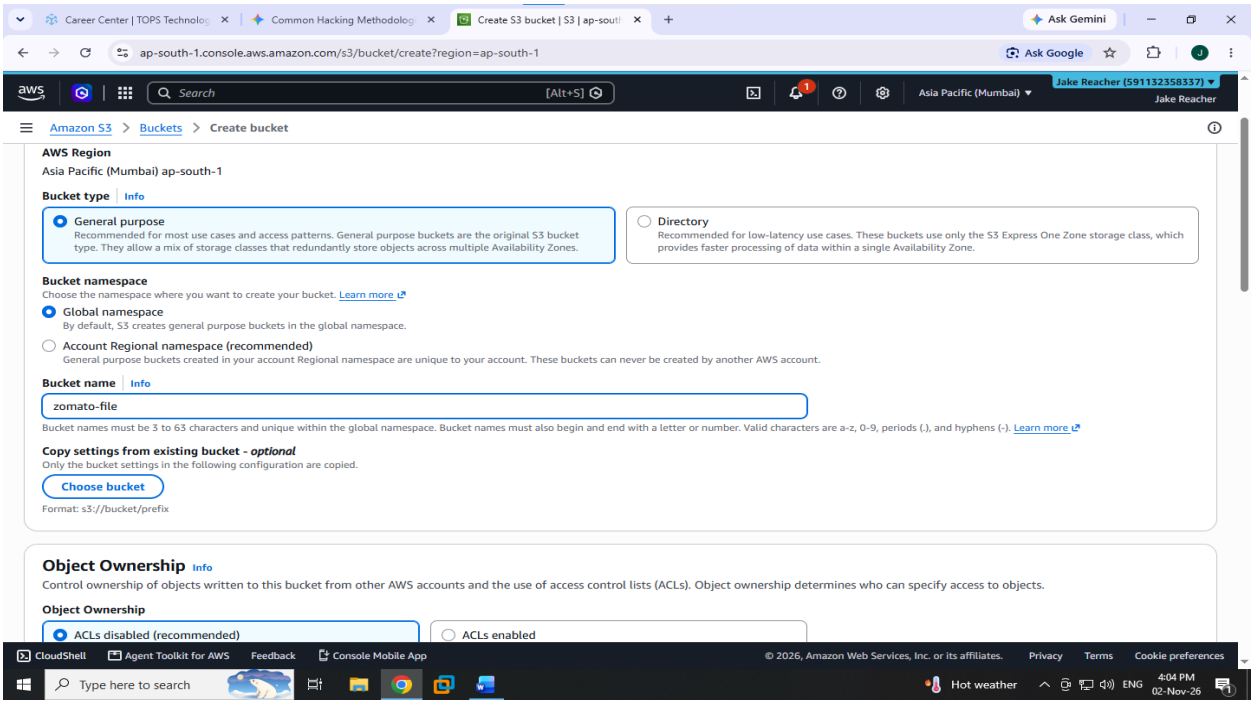
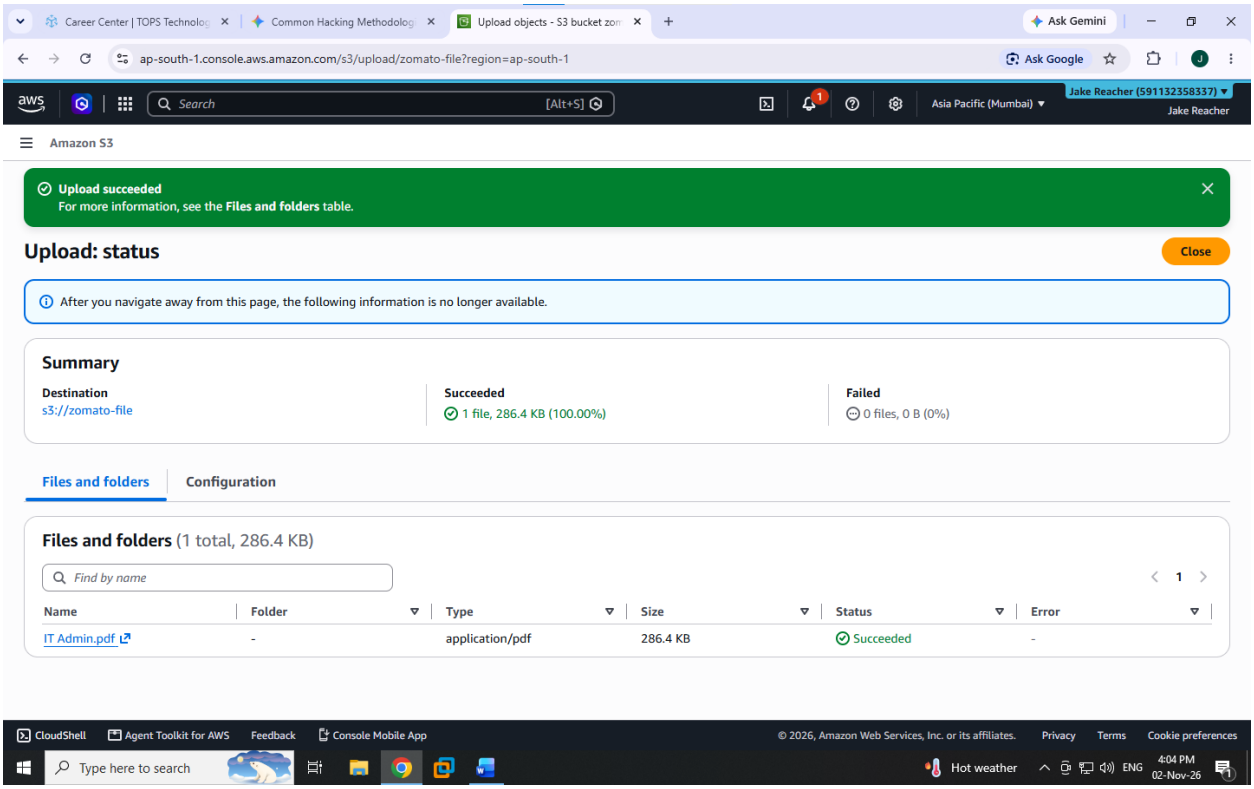
# ASSIGNMENT-14 : KEY CLOUD CONCEPTS

**GUIDED BY : VIPUL MISTRI**

## Q.1 Create a new S3 bucket on AWS named after your favorite food delivery app (e.g., swiggy-files), and upload any one image or PDF file from your computer to this bucket.

## ANS :



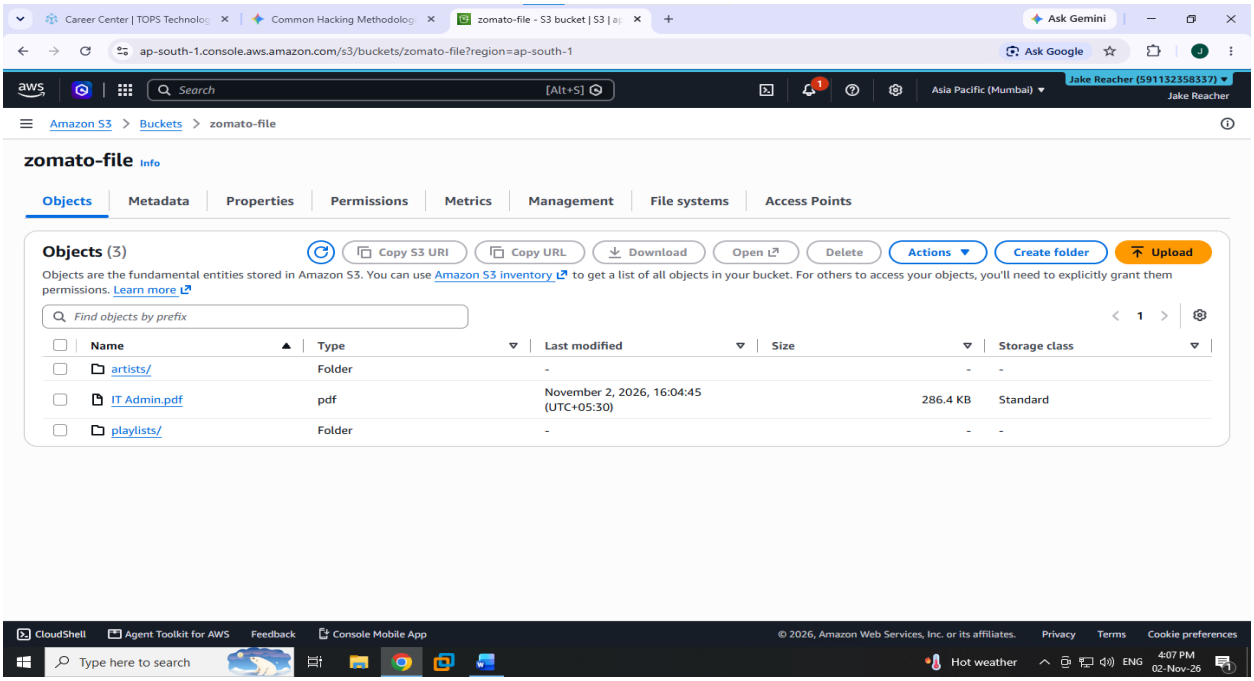



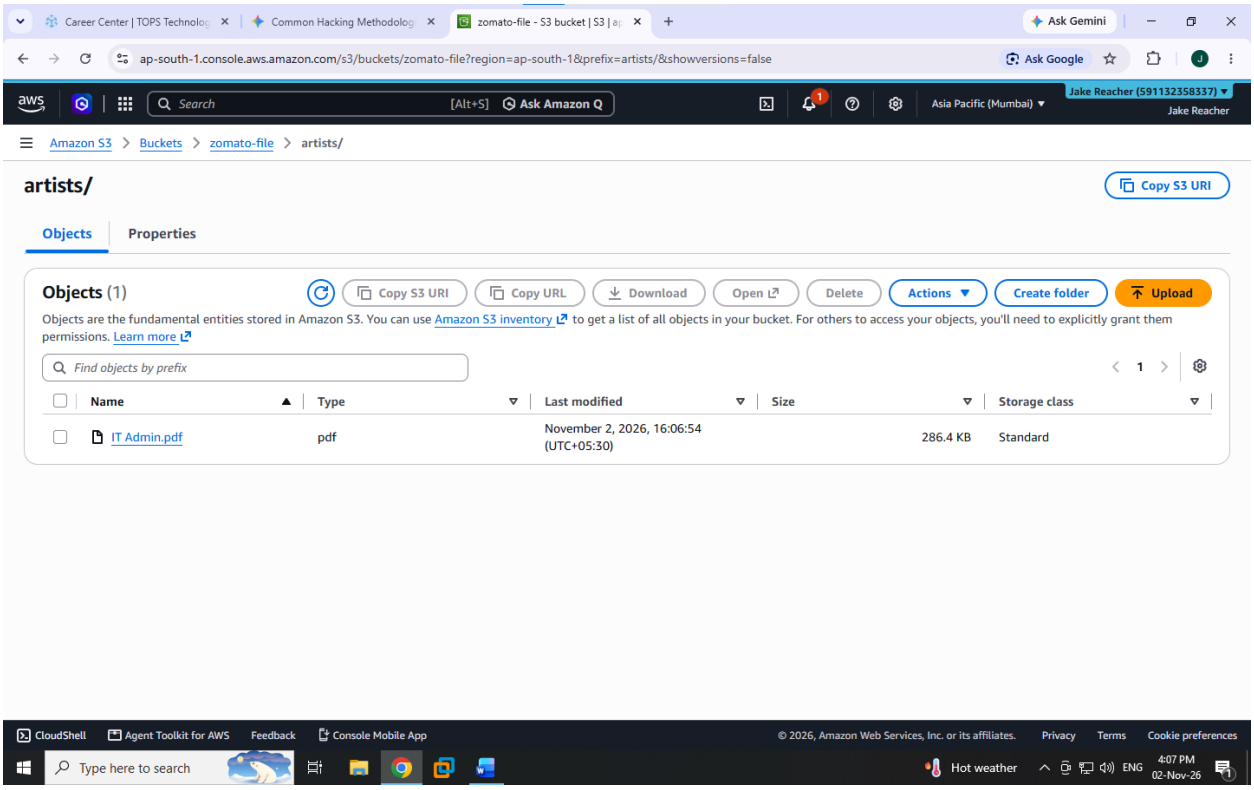
# ASSIGNMENT-14 : KEY CLOUD CONCEPTS

**GUIDED BY : VIPUL MISTRI**

## Q.2 Set up a folder structure inside your S3 bucket that mimics a music streaming app like Spotify: create folders named 'albums', 'artists', and 'playlists', and upload at least one sample file into each folder.

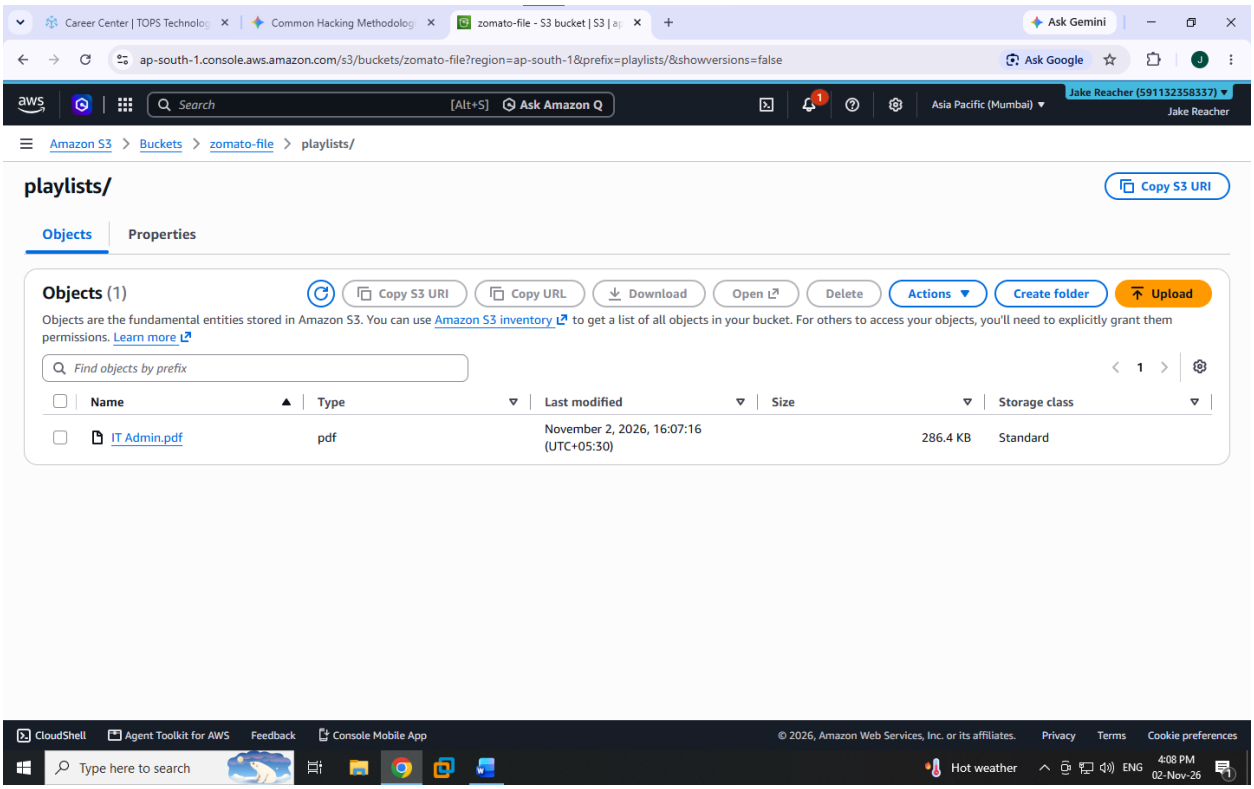
## ANS :



# ASSIGNMENT-14 : KEY CLOUD CONCEPTS

**GUIDED BY : VIPUL MISTRI**




**Q.3 Configure an IAM user named 'zomatoUploader' and grant it permission to only upload and list files in your S3 bucket, but not delete or download files. Hint: Use the AWS IAM console to attach a custom policy with s3:PutObject and s3:ListBucket permissions only.**

**ANS :** To configure an IAM user with strictly controlled permissions to **only upload** (PutObject) **and list** (ListBucket) files in your S3 bucket without allowing them to delete or download, follow these step-by-step instructions using the AWS Management Console (GUI):

**Step 1: Create the IAM User** (zomatoUploader)

1. Log in to the AWS Management Console.

2. In the top search bar, type **IAM** and open the Identity and Access Management service.

3. In the left-hand navigation pane, click on **Users**, then click **Create user** in the top right.

4. **Specify user details:**

    - **User name:** zomatoUploader

    - *Optional:* Leave "Provide user access to the AWS Management Console" unchecked (since programmatic upload scripts/CLI are typically used for automated tasks), or check it if you need a console login. Click **Next**.

5. **Set permissions:**

    - Select **Attach policies directly**.

    - Leave them unselected for a moment because we will create a custom least-privilege policy. Click **Next**, then click **Create user**.

**Step 2: Create a Custom Least-Privilege IAM Policy**

1. In the IAM dashboard left-hand menu, click on **Policies**, then click **Create policy**.
2. Select the **JSON** tab and replace the default code with the following strict policy JSON document (make sure to replace zomato-files-lab with your actual bucket name):

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
        {
            "Sid": "AllowListBucket",
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket"
            ],
            "Resource": "arn:aws:s3:::zomato-files-lab"
        },
        {
            "Sid": "AllowUploadObjects",
            "Effect": "Allow",
            "Action": [
                "s3:PutObject"
            ],
            "Resource": "arn:aws:s3:::zomato-files-lab/*"
        }
    ]
}
```

3. Click **Next**, name your policy S3UploadAndListOnlyPolicy, and click **Create policy**.

**Step 3: Attach the Custom Policy to** zomatoUploader

1. Go back to **Users** in the IAM menu and click on zomatoUploader.
2. Under the **Permissions** tab, click **Add permissions** → **Attach policies directly**.
3. Search for S3UploadAndListOnlyPolicy, check the box next to it, and click **Add permissions**

**ASSIGNMENT-14 : KEY CLOUD CONCEPTS**

**GUIDED BY : VIPUL MISTRI**

**Q.4 Test your IAM setup by logging in as 'zomatoUploader' and trying to upload a file and then delete a file from your S3 bucket; note which actions succeed and which are denied.**

**ANS** : As a cloud-hosted AI assistant, I don't have an active AWS account, local terminal environment, or the zomatoUploader IAM credentials to execute actual cloud API calls against an S3 bucket.

However, since you configured zomatoUploader with the least-privilege policy allowing only s3:ListBucket and s3:PutObject, here is exactly what you will observe when you test this setup in your local environment using the AWS CLI:

Expected Test Results

1. Testing the Upload Action (s3:PutObject) — SUCCESS

- Command to Run:

Bash

aws s3 cp test-image.jpg s3://zomato-files-lab/albums/ --profile zomatoUploader

- Expected Result: The file uploads successfully, and the CLI returns an upload progress bar and completion hash. This succeeds because s3:PutObject is explicitly allowed in your custom IAM policy.

2. Testing the List Action (s3:ListBucket) — SUCCESS

- Command to Run:

Bash

aws s3 ls s3://zomato-files-lab/albums/ --profile zomatoUploader

- Expected Result: The CLI lists all objects inside the folder. This succeeds because s3:ListBucket is explicitly permitted.

3. Testing the Delete Action (s3:DeleteObject) — DENIED (Access Denied)

- Command to Run:

Bash

aws s3 rm s3://zomato-files-lab/albums/test-image.jpg --profile zomatoUploader

- Expected Result: The operation fails with an AccessDenied error message from AWS.

- Why it fails: AWS IAM follows a default-deny principle. Because s3:DeleteObject (and s3:GetObject) were not included in your custom policy, AWS automatically blocks the request, successfully enforcing your security constraint.

**Q.5 Write a short note explaining how IAM roles and policies in AWS help protect user data in cloud apps like Paytm or PhonePe, and give one real-world example of a security risk if IAM is not configured properly.**

ANS : IAM (Identity and Access Management) roles and policies act as the foundational security gatekeepers for cloud applications like Paytm or PhonePe. **IAM policies** are JSON documents that define *what* actions are allowed or denied (e.g., read, write, delete), while **IAM roles** are secure, temporary identities assumed by applications, services, or trusted users rather than sharing permanent master credentials. Together, they enforce the **Principle of Least Privilege**, ensuring that backend microservices, database connectors, and payment gateways only have access to the exact resources they need—preventing unauthorized exposure of sensitive user financial data, transaction logs, and KYC documents stored in cloud buckets or databases.

**Real-World Security Risk: Overly Permissive S3 Bucket Access**

If an IAM policy or role is misconfigured—such as granting s3:* (wildcard full access) or public read/write permissions to a backend service role handling user documents—it creates a severe vulnerability.

- **The Risk:** An attacker who compromises a low-level web server or an unauthenticated API endpoint could exploit the overly permissive IAM role to list, download, or tamper with millions of users' encrypted transaction histories, bank statements, or Aadhaar/PAN KYC scans stored in cloud storage.

- **Real-World Impact:** This type of misconfiguration is a primary driver behind major cloud data leaks, resulting in massive financial fraud, identity theft, and severe regulatory penalties. Proper IAM scoping ensures that even if an application component is breached, the attacker's access remains strictly isolated.